📐 Program Organization: Structured vs. Object-Oriented

> **Structured Programming** – organizes code **around functions/procedures**.
> **Object-Oriented Programming (OOP)** – organizes code **around data** (objects).

- **Structured**: any function can operate on any data type.
- **OOP**: only the functions (methods) permitted by a data type may act on that data; data controls access.

⚙️ Procedure-Oriented vs. Object-Oriented Programming

Feature	Procedure-Oriented (PO)	Object-Oriented (OO)
Program division	Functions	Objects
Approach	Top-down	Bottom-up
Access specifiers	None	private, public, protected
Data security	Less secure	Supports data hiding
Extensibility	Hard to add new data/functions	Easy to add new data/functions
Overloading	Not supported	Supported
Inheritance / data hiding	Not supported	Supported
Emphasis	Functions > Data	Data > Functions
Modeling	Unreal-world	Real-world entities
Typical size	Medium programs	Large/complex programs
Abstraction	Procedure abstraction	Data abstraction
Code reuse	No	Yes
Examples	C, FORTRAN, Pascal	C++, Java, Python

📦 Encapsulation & Data Encapsulation

> **Encapsulation** – binds code and the data it manipulates into a single self-contained unit (an *object*), creating a clear boundary between internal implementation and external use.

- **Private members** – accessible only inside the class.
- **Public members** – provide controlled access to private parts.

> **Data Encapsulation** – combines data and its manipulating functions into a **class**, restricting direct data access and enabling data hiding.

🔀 Polymorphism

> **Polymorphism** – one interface controls access to a family of actions; the specific action is chosen based on context.

Types in C++

Type	Mechanism
Compile-time	Function and operator **overloading**
Run-time	**Virtual functions** and inheritance

- **Overloading** is a form of compile-time polymorphism: the same function/operator behaves differently for different argument types.

Example: Stacks with Unified Interface

```
// No code snippet needed beyond description; the idea is three stack classes
// each providing push() and pop() with the same names.
```

Advantages

- Reduces complexity.
- Programmer uses a general interface; compiler selects the correct implementation.

🌳 Inheritance

> **Inheritance** – creates a **derived class** from an existing **base class**, transferring the base's data members and member functions.

- **Base class** = parent / super class.
- **Derived class** = child / sub class.

Significance

- Enables **code reuse**.
- Provides hierarchical classification.
- Reduces code size and improves structure.
- Allows creation of specific instances from general concepts.

🎭 Data Abstraction

> **Data Abstraction** – defines *what* a type does without exposing *how* it does it.

- Allows user-defined types that hide implementation details.
- Simplifies interaction by exposing only necessary features.

⏩ I/O Operators (<< and >>)

- << – **insertion** (output) operator for cout.
- >> – **extraction** (input) operator for cin.

Both handle all built-in types and can be **chained**:

```
#include <iostream>
using namespace std;

int main() {
    float f;
    char str[80];
    double d;
    cout << "Enter two floating point numbers: ";
    cin >> f >> d;
    cout << "Enter a string: ";
    cin >> str;          // stops at first whitespace
    cout << f << " " << d << " " << str;
    return 0;
}
```

Limitation – >> stops reading a string at the first whitespace, so "This is a test" reads only "This".

📍 Local Variable Declarations

- Variables may be declared **anywhere** inside a block, typically **at first use**.
- **Benefits**: tighter encapsulation, reduced scope, clearer intent, easier maintenance.
- **Considerations**: in very short functions, top-of-block declarations may still be acceptable.

📊 Variable Declarations & Initializations

- Declarations can be interleaved with statements.
- **Good style**: still group related declarations at the top of a block.

For-loop Counter Example

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i < 5; ++i)
        cout << i << endl;
    // i is not accessible here
    return 0;
}
```

Global Variable Initialization

```
int rows = 5;
int cols = 6;
int size = rows * cols;   // valid: any expression allowed
```

Array of Pointers Initialization

```
void example() {
    int n1, n2, n3;
    int *nptr[] = { &n1, &n2, &n3 };   // array of pointers
}
```

❌ "Default-to-int" Rule

- **Removed** from standard C++.
- Omitting a type in a declaration now yields a **compiler error**.

```
// Error in standard C++:
func(int i) { return i*i; }   // missing return type

// Correct:
int func(int i) { return i*i; }
```

✅ bool Data Type

- Stores **true** (1) or **false** (0).
- Implicitly converts: non-zero → true, zero → false.
- Used by logical operators (&&, ||, !) and relational operators (==, !=, <, …).
- Enables clear conditional expressions in if, while, for, etc.

📚 New-style vs. Old-style Headers

Style	Syntax	Remarks
Old-style	#include <iostream.h>	.h extension, deprecated.
New-style	#include <iostream>	No extension; preferred.
C headers in C++	#include <cstdio> (instead of stdio.h)	Add c prefix, drop .h.

Recommendation – always use new-style headers in modern C++ code.

🔗 void* Usage

- **C**: implicit conversion between any object pointer and void*.
- **C++**: **explicit** cast required.

```
int N;
int *P = &N;
void *Q = P;                  // illegal in C++
void *R = (void *)P;          // legal in C++
```

🏗️ struct and Tags

> **struct** – user-defined type grouping heterogeneous members.

```
struct MyType {
    int A;
    float B;
};
MyType v;   // tag acts as a type name (no `struct` keyword needed)
```

- **Tag** = the name (MyType) that identifies the structure type.

🔢 enum Differences (C vs. C++)

> **enum** – named integral constants.

```
enum Color { red, blue, green };
```

- **C**: enums behave as plain integers (allow implicit conversion, arithmetic).
- **C++**: strongly typed; no implicit conversion to/from int, preventing unsafe operations.

📦 Namespaces & using namespace std;

- **Namespaces** prevent name collisions by providing distinct scopes.
- Standard library resides in the **std** namespace.

```
using namespace std;   // makes std::cout, std::cin, etc., accessible without prefix
// Equivalent without using:
std::cout << "Hello";
```

🏛️ Classes

> **Class** – blueprint for objects, encapsulating data and behavior.

```
class Stack {
private:
    int *data;
    int top;
public:
    Stack();                // constructor
    void push(int x);
    int pop();
    ~Stack();               // destructor
};
```

- **Default access**: private.
- **Member functions** defined outside class use the scope resolution operator ::.

```
void Stack::push(int x) {
    // implementation
}
```

- **Object creation**: Stack s; creates an instance; Stack *p = new Stack; allocates dynamically.

⚖️ Struct vs. Class

Aspect	struct	class
Default access	public	private

Typical use	Passive data containers	Full OOP design
Inheritance default	public	private
Supported features	Same as class (functions, constructors, etc.)	Same as struct

---

🛠️ Constructors & Destructors

Constructors

- Same name as the class, **no return type**.
- Called automatically on object creation.
- Can be **default**, **parameterized**, **copy**, or have **default arguments**.

```
class Hello {
public:
    Hello();           // default constructor
    Hello(int x);      // parameterized
    Hello(const Hello&); // copy constructor
    ~Hello();          // destructor
};
```

Destructors

- Same name preceded by ~.
- No parameters, no return type.
- Invoked automatically when an object's lifetime ends.

```
Hello::~Hello() {
    // cleanup code
}
```

Default Constructor Behavior

- If not defined, the compiler supplies one that **zero-initializes** built-in members and **calls** default constructors of member objects.

Parameterized Constructors

```
class Rectangle {
    int w, h;
public:
    Rectangle(int a, int b) : w(a), h(b) {}
```

```
};
Rectangle r(3,4);   // uses parameterized constructor
```

Copy Constructors

```
class Point {
public:
    double x, y;
    Point(const Point& p) : x(p.x), y(p.y) {}
};
Point a;
Point b = a;   // copy constructor invoked
```

Constructors with Default Arguments

```
class X {
public:
    X(int a = 7, int b = 9);
};
X obj;           // behaves as default constructor
X obj2(5);       // b uses default value 9
```

🚀 Dynamic Initialization of Objects

> **Dynamic initialization** – object is initialized at runtime using values supplied during execution.

```
class Person {
    char *name;   // dynamic
    int age;
    static int id;
public:
    Person(const char *n, int a, int i) {
        int len = strlen(n);
        name = new char[len + 1];
        strcpy(name, n);
        age = a;
        id = i;
    }
    ~Person() { delete[] name; }
};
```

- Constructor receives user input, allocates memory with new, and initializes members accordingly.

✏️ Comments in C++

- **Single-line comment**: // this is a comment
- **Multi-line comment**: /* comment spanning multiple lines */

⚙️ Operator Categories (C++)

Category	Symbols	Brief Description
Arithmetic	+ - * / %	Basic math operations
Relational	== != < > <= >=	Compare values, yield bool
Logical	&& || !	Combine boolean expressions
Assignment	= += -= *= /= %=	Assign values
Inc/Dec	++ --	Prefix/postfix increment/decrement
Conditional	?:	Ternary if-else
Bitwise	& | ^ ~ << >>	Operate on individual bits
Scope Resolution	::	Qualify names (e.g., Class::member)
Member Access	. ->	Access members via object or pointer
I/O	<< >>	Stream insertion/extraction (cout, cin)